



Lista de Exercícios - Programação Orientada a Objetos em Java – 25/09/2026

Classes, objetos, atributos, métodos, parâmetros, visibilidade e encapsulamento

Orientações gerais:

- Crie uma classe de domínio para cada problema e uma classe com main para testar os objetos.
- Use apenas recursos básicos de Java: classes, atributos, métodos, parâmetros, if, return, criação de objetos com new e saída com `System.out.println`.
- Quando solicitado, deixe os atributos como `private` e acesse ou altere o estado dos objetos por meio de métodos públicos.
- Teste situações válidas e inválidas para observar como os métodos protegem o estado dos objetos.

QUESTÃO 1

Cadastro simples de aluno

Uma escola deseja representar alunos em um pequeno programa Java. Neste primeiro exercício, o objetivo é perceber a diferença entre classe e objeto, além de praticar a criação de atributos, métodos simples e chamadas por meio de uma variável de referência.

Enunciado

Crie uma classe `Aluno` para representar um estudante. A classe deve possuir os atributos `matricula`, `nome`, `curso` e `notaFinal`.

Nesta questão, faça uma primeira versão com os atributos públicos, apenas para praticar o acesso direto e observar como isso funciona. Em seguida, crie métodos para exibir os dados do aluno e informar se ele foi aprovado.

Requisitos

- Crie a classe `Aluno` com os atributos públicos indicados no enunciado.
- Crie o método `exibirDados()`, que mostre matrícula, nome, curso e nota usando `System.out.println`.
- Crie o método `verificarAprovacao()`, que mostre se o aluno foi aprovado ou reprovado. Considere aprovado quem tiver nota maior ou igual a 7.0.
- Crie uma classe `ProgramaAluno` com o método `main`.
- No `main`, crie dois objetos do tipo `Aluno`, atribua valores diferentes aos seus atributos e chame os métodos de cada objeto.

Arquivos sugeridos: `Aluno.java` e `ProgramaAluno.java`

QUESTÃO 2

Conta de energia elétrica

Uma concessionária precisa calcular o valor mensal de contas de energia. Cada conta possui dados próprios, como consumidor, consumo e valor do kWh. O exercício trabalha estado, comportamento e encapsulamento básico.

Enunciado

Crie uma classe `ContaEnergia` que represente a conta mensal de um consumidor. Agora os atributos devem ser privados, e o acesso ao estado do objeto deve ocorrer por métodos públicos.

Atributos privados

- `numeroInstalacao`
- `nomeCliente`
- `consumoKwh`
- `valorKwh`

Métodos públicos

- `definirCliente(String numero, String nome)`: define o número da instalação e o nome do cliente.
- `definirValorKwh(double valor)`: altera o valor do kWh somente se o valor for maior que zero.
- `registrarConsumo(double kwh)`: registra o consumo somente se o valor informado não for negativo.
- `calcularValor()`: retorna o valor da conta, multiplicando consumo por valor do kWh.
- `exibirResumo()`: imprime os dados da conta e o valor calculado.

Testes no método main

1. Crie dois objetos `ContaEnergia`.
2. Defina clientes e consumos diferentes para cada objeto.
3. Chame `exibirResumo()` para cada conta.
4. Tente registrar um consumo negativo e verifique se o estado do objeto permanece válido.

Arquivos sugeridos: `ContaEnergia.java` e `ProgramaContaEnergia.java`

QUESTÃO 3

Produto em estoque

Uma loja precisa controlar produtos disponíveis para venda. O sistema deve impedir operações inválidas, como vender uma quantidade maior que o estoque ou cadastrar preço negativo.

Enunciado

Crie uma classe `Produto` com atributos privados e métodos públicos que controlem corretamente as alterações do estoque.

Atributos privados

- `codigo`
- `nome`
- `preco`
- `quantidadeEstoque`

Métodos públicos obrigatórios

- `cadastrar(String codigo, String nome, double precoInicial)`: define os dados iniciais do produto. O preço inicial deve ser aceito apenas se for maior que zero.
- `repor(int quantidade)`: aumenta o estoque somente se a quantidade for positiva.
- `vender(int quantidade)`: diminui o estoque somente se a quantidade for positiva e houver estoque suficiente. O método deve retornar `true` se a venda ocorrer e `false` caso contrário.
- `alterarPreco(double novoPreco)`: altera o preço somente se o novo preço for maior que zero.
- `exibirDados()`: mostra o código, nome, preço e quantidade em estoque.

Testes no método main

1. Crie pelo menos dois produtos.
2. Faça reposições com quantidades válidas.
3. Realize uma venda válida e mostre o resultado com `System.out.println`.
4. Tente vender uma quantidade maior que o estoque.
5. Tente alterar o preço para um valor negativo.
6. Exiba o estado final dos objetos.

Arquivos sugeridos: `Produto.java` e `ProgramaProduto.java`

QUESTÃO 4

Controle de veículo em uma viagem

Um aplicativo simples de viagens deseja registrar informações de veículos. Cada veículo possui seu próprio estado, e chamadas de métodos em um objeto não devem alterar automaticamente outro objeto.

Enunciado

Crie uma classe `Veiculo` que permita controlar a velocidade e a quilometragem percorrida por um veículo. Use atributos privados e métodos públicos com parâmetros para modificar o estado do objeto.

Atributos privados

- `placa`
- `modelo`
- `velocidadeAtual`
- `quilometragem`

Métodos públicos obrigatórios

- `cadastrar(String placa, String modelo)`: define os dados iniciais do veículo.
- `acelerar(double valor)`: aumenta a velocidade se o valor for positivo.
- `frear(double valor)`: diminui a velocidade se o valor for positivo. A velocidade nunca deve ficar abaixo de zero.
- `registrarViagem(double km)`: aumenta a quilometragem se a distância for positiva.
- `exibirPainel()`: imprime placa, modelo, velocidade atual e quilometragem.

Testes no método main

1. Crie dois objetos `Veiculo`, por exemplo `carro1` e `carro2`.
2. Cadastre dados diferentes para cada veículo.
3. Chame `acelerar`, `frear` e `registrarViagem` em cada objeto com valores diferentes.
4. Mostre que o estado de `carro1` é independente do estado de `carro2`.
5. Teste uma frenagem maior que a velocidade atual e confirme que a velocidade final fica em zero.

Arquivos sugeridos: `Veiculo.java` e `ProgramaVeiculo.java`

QUESTÃO 5

Cliente e conta bancária

Um banco deseja representar clientes e contas. O cliente tem seus próprios dados, enquanto a conta controla saldo, depósito, saque e rendimento. O objetivo é praticar mais de uma classe, criação de objetos, chamadas de métodos entre objetos e encapsulamento.

Enunciado

Crie duas classes de domínio: `Cliente` e `ContaBancaria`. A classe `ContaBancaria` deve guardar uma referência para um objeto `Cliente`, representando o titular da conta.

Classe `Cliente`

Atributos privados:

- `nome`
- `cpf`
- `telefone`

Métodos públicos:

- `cadastrar(String nome, String cpf, String telefone)`
- `exibirDados()`
- `getNome()`, para permitir que a conta mostre o nome do titular sem expor diretamente o atributo.

Classe `ContaBancaria`

Atributos privados:

- `numero`
- `titular`, do tipo `Cliente`
- `saldo`

Métodos públicos:

- `abrirConta(String numero, Cliente titular)`: associa a conta a um cliente.
- `depositar(double valor)`: aceita apenas valores positivos.
- `sacar(double valor)`: permite saque apenas se o valor for positivo e menor ou igual ao saldo. Retorne `true` ou `false`.
- `renderConta(double percentual)`: aumenta o saldo com base em um percentual positivo. Exemplo: se o saldo é 1000 e o percentual é 10, o novo saldo será 1100.
- `exibirExtrato()`: mostra número da conta, nome do titular e saldo.

Testes no método main

1. Crie dois objetos Cliente.
2. Crie duas contas bancárias, cada uma associada a um cliente diferente.
3. Faça depósitos e saques nas contas.
4. Tente realizar um saque maior que o saldo e imprima se a operação foi aceita ou recusada.
5. Aplique rendimento em uma das contas.
6. Exiba os extratos finais e os dados dos clientes.

Restrições importantes

- O atributo saldo não pode ser público.
- O saldo não pode ser alterado diretamente no main.
- A única forma de mudar o saldo deve ser pelos métodos depositar, sacar e renderConta.
- Use `System.out.println` para mostrar as mensagens de teste.

Arquivos	sugeridos:	Cliente.java,	ContaBancaria.java	e
		ProgramaBanco.java		